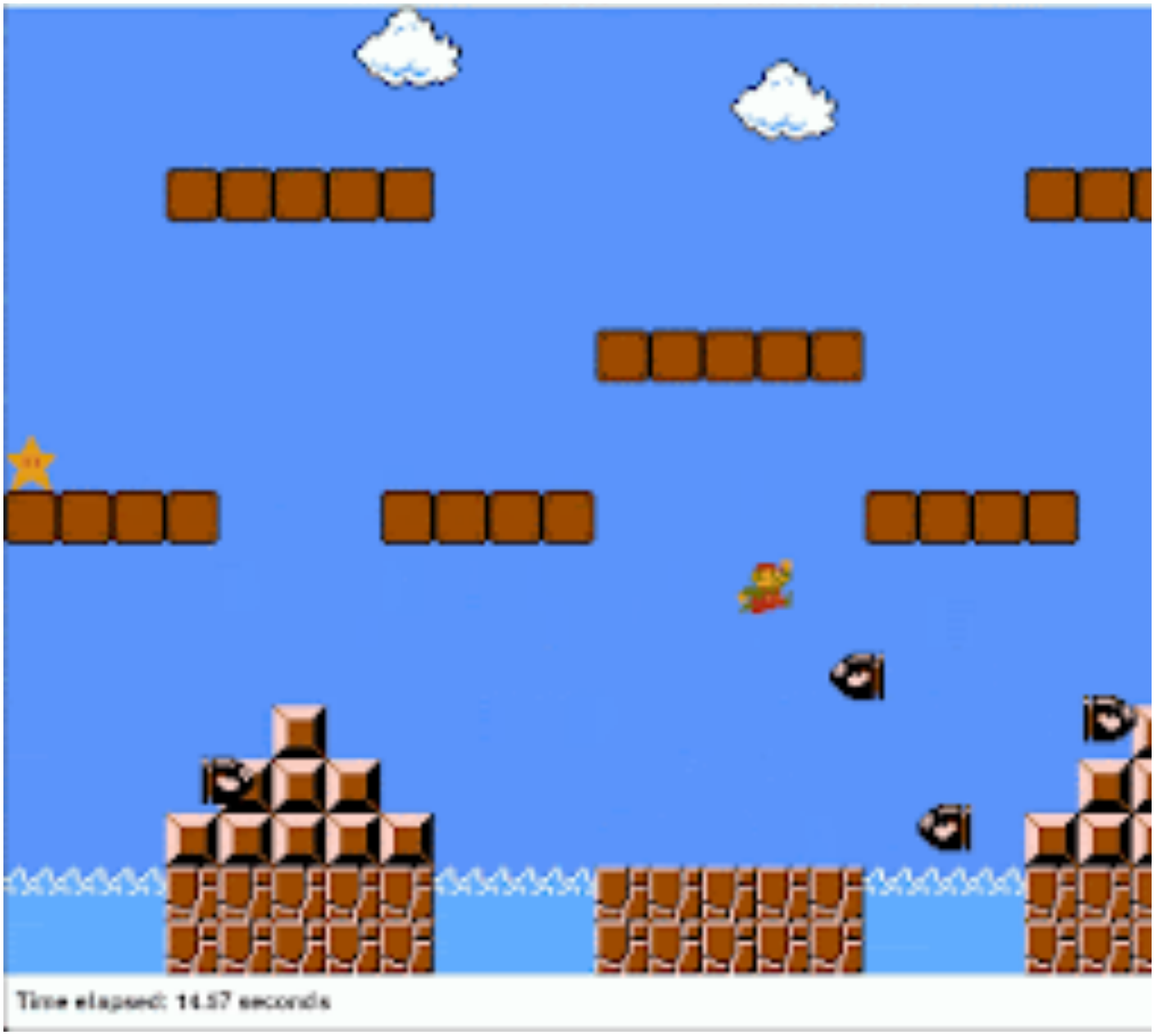
2026-02 KNU Advanced RL Competition Round 2

Bullet Bills!



두 번째 RL Competition은 화면 양 옆에서 등장하는 떨어지는 킬러Bullet Bill을 피해 최대한 오랫동안 살아남는 Bullet Bills입니다!

Purpose

플레이어는 마리오를 좌우 이동 및 점프 조작을 통해 양 옆에서 등장하는 킬러를 피해야 합니다. Avoid Blurps와 마찬가지로 킬러는 시간이 갈수록 더 많이 등장하고, 등장 위치와 속도 또한 무작위로 변합니다. 여러분은 최대 3분 동안 킬러의 공격을 피해야 합니다. 다행히, 여러분에게 무적을 선사하는 별도 있습니다.

Prerequisites

Python 3.12.x

Bullet Bills은 Python 3.12 버전에서 동작하도록 구현되었습니다 (그 외의 버전에서는 동작을 보장할 수 없습니다)

Installation

Bullet Bills 환경은 다음과 같이 설치할 수 있습니다:

```
pip install -U kymnasium
```

그 외 실행에 필요한 `gymnasium`, `pygame`, `numpy` 등의 라이브러리는 의존성으로 함께 설치되므로 별도 설치할 필요는 없습니다.

※ 이전에 이미 설치했더라도, -U 옵션을 통해 최신 버전 (1.2.9)으로 업데이트 하셔야 합니다!

Environment

Generation

Bullet Biills 환경은 일반적인 gymnasium 환경처럼 다음과 같이 생성할 수 있습니다:

```
import gymnasium as gym


env = gym.make(
  # Bullet Bills의 ID
  id='kymnasium/BulletBill-Continuous-Hard-Stage-2',
  # "human": 메인 스레드에서 게임 화면 렌더링,
  # "rgb_array": 백그라운드에서 게임 화면 렌더링
   # "none": 게임 화면 렌더링 없이 내부 로직만 모델링; 에이전트 훈련에
             활용
```

```
  render_mode='human',
  # "True" 일 시 BGM 재생; 오직 render_mode = "human"일 때만 동작
  bgm=True,
   # 랜덤 시드
   seed=42
)
```

Observation Space

에이전트는 마리오, 킬러, 별의 속성들을 *mario*, *bullet_bills*, *stars*라는 키를 갖고 Numpy 행렬을 값으로 가지는 딕셔너리로 알 수가 있습니다.

```
{
  "mario": [좌측 좌표, 위쪽 좌표, 우측 좌표, 아래쪽 좌표, X축 속도, Y축 속도,
무적 여부 (1일시 무적 모드; 아닐 시 0), 점프 여부 (1일시 점프 중; 아닐 시 0)],
   "bullet_bills":  [
     [좌측 좌표, 위쪽 좌표, 우측 좌표, 아래쪽 좌표, X축 속도],
     [좌측 좌표, 위쪽 좌표, 우측 좌표, 아래쪽 좌표, X축 속도],
     ...
  ],
   "stars":  [
     [좌측 좌표, 위쪽 좌표, 우측 좌표, 아래쪽 좌표],
     [좌측 좌표, 위쪽 좌표, 우측 좌표, 아래쪽 좌표],
     ...
  ]
}
```

*mario*는 플레이어가 조작하는 마리오의 속성을 8개의 원소를 가지는 Numpy 행렬로 표시합니다.

*bullet_bills*은 [30, 5] Numpy 행렬로 제공되며, 각 행마다 화면에 표시된 킬러의 속성을 5개의 원소로 표현합니다. 화면에 표시되지 않거나 마리오에게 잡힌 킬러의 경우 모든 원소의 값이 0이며, 좌표는 화면 좌측 최상단이 각각 0, 0입니다.

*stars*는 [3, 4] Numpy 행렬로 제공되며, 역시 각 행마다 화면에 표시된 별의 속성을 4개의 원소로 표현합니다. 화면에 표시되지 않은 별의 경우 모든 원소의 값이 0입니다.

Action Space

에이전트가 움직이도록 하기 위해서는 *Env.step()* 호출 시 *np.array([moving_force, jump_force])* 형태로 전달해야 합니다. 예를 들어, *Env.step(np.array([0.2, 0.3]))* 형태입니다.

- *moving_force*: 마리오를 좌우로 이동시키기 위해 가할 힘으로, [-1, 1] 사이의 범위입니다.
- *jump_force*: 마리오를 점프시키기 위해 가할 힘으로, [0, 1] 사이의 범위입니다.

안타깝게도, 2단 점프는 안 됩니다. 즉, 점프 중에 또 점프를 할 수는 없습니다.

Reward Space

어떤 행동이든 보상의 양을 0으로 설정했습니다. 여러분들이 자유롭게 보상을 설계해보세요!

Info

현재까지 경과한 시간과 게임 상태를 다음과 같이 반환합니다:

```
{
  "time_elapsed": XXXX,
  "game_state": XXXX
  # "init": 초기 상태
  # "playing": 일반적인 플레이 중
   # "game_over": Blurp에 닿아서 게임 종료
   # "cleared": 주어진 시간 동안 살아남음
}
```

Episode End

킬러와 마리오가 충돌하거나, 3분 동안 킬러를 피한 경우 에피소드가 종료되며 반환되는 값은 다음과 같습니다:

- 킬러가 마리오가 충돌한 경우:
  - terminated = True, truncated = False
  - info['*game_state*'] = '*game_over*'
- 3분 동안 킬러를 피한 경우:
  - terminated = True, truncated = False
  - info['*game_state*'] = '*cleared*'

Manual Play

kymnasium.bullet_bills.ManualPlayWrapper(env_id: str, debug: bool, **kwargs).play(seed)를 활용해 지난 환경과 마찬가지로, 직접 플레이해 볼 수도 있습니다!

- **env_id**: *gymnasium.make()* 등을 통해 환경을 불러올 때 사용하는 환경의 id를 의미합니다.
- **debug**: *True*일 경우 각종 로그를 출력합니다
- **seed**: Random Seed를 설정합니다.

조작 방법은 다음과 같습니다:

- 방향키 ←: 마리오가 좌측으로 조금 이동합니다.
- 방향키 →: 마리오가 우측으로 조금 이동합니다.
- 스페이스 바: 마리오가 점프합니다.

Evaluation

Competition (20%)

Code

RL Competition은 수업 시간 중 본인의 기기 (노트북, 원격 연결 등)에서 다음의 코드를 실행하여 플레이한 결과로 결정됩니다 (별도의 실행 코드는 인정하지 않습니다):

```
import kymnasium as kym

'''
여러분이 정의하고 학습시킨 에이전트를 불러오는 코드를 넣으세요
예. agent = YourAgent.load('some-path.pkl')
'''
agent = ...

kym.evaluate(
   env_id='kymnasium/BulletBill-Continuous-Hard-Stage-2',
   agent=agent,
   render_mode='human',
   bgm=True,
    seed=... # 시연 당일 설정
)
```

Grading Criteria



킬러를 피해 최대한 오래 버티는 게 목표이며, **총 2번 시도 중 가장 오래 버틴 기록** 순으로 순위를 매깁니다. 순위에 따른 점수는 다음과 같습니다:

- 1위: 20%
- 2위: 19%
- 3위: 18%
- 4위: 17%
- 5위: 16%
- 6위: 15%
- 7위: 14%

- 8위: 13%
- 9위: 12%
- 10위: 11%
- 11위: 10%
- 12위: 9%
- 13위: 8%
- 14위: 7%
- 15위: 6%
- 16위: 5%

Presentation (10%)

Purpose

Competition 의 결과 뿐만 아니라, 여러분이 환경을 어떻게 분석했고, 어떤 강화학습 방법을 선택했으며, 무엇이 성공하거나 실패했는지를 공유하여 다른 학생들이 더 좋은 에이전트를 구현할 수 있도록 돕는 5분 내외의 발표도 하셔야 합니다.

Grading Criteria

- 환경 및 문제 분석 (2%): 주어진 환경을 해결하기 위한 상태, 행동, 보상 등의 처리 방법
- 강화학습 방법의 이해와 선택 근거 (3%): 알고리즘, 모델 구조, 탐험 방법, 주요 하이퍼파라미터의 선택 근거와 대안과의 차이
- 다른 학생에게 제공하는 실질적 도움 (3%): 다른 학생이 적용할 수 있는 구체적인 개선 방법과 적용 조건, 예상 효과, 구현 시 주의점, 재현을 위한 설정 및 학습량 등
- 한계 및 개선점 (1%): 현재 적용한 방법의 한계와 그 원인을 분석하고, 개선을 위한 아이디어 제공
- 전달력과 시간 활용 (1%): 5분 안에 핵심을 논리적이고 명확하게 전달

Peer Evaluation Only

발표는 교수가 평가하는 대신 수업을 듣는 다른 학생들이 평가합니다. 발표의 목적은 다른 학생들이 ***"이 발표를 듣고 나서 내가 구현하거나 실험해 볼 수 있는 구체적인 개선 방법이 생겼는가?"***를 느꼈는지 평가하기 위함이기 때문입니다. 따라서, ***"ChatGPT가 그랬는데요"*** 등의 내용은 학생들에게 도움이 되지 않으므로 허용되지 않습니다.

Submission

이루리에 제출해야 하는 파일은 총 세 개입니다.

Python for Training Your Own Agent (*.py)

아래 작성 예시를 복사하여, 자신만의 에이전트를 정의하고 이 에이전트를 훈련하는 코드를 구현한 후 제출하세요.

```
import gymnasium as gym
import kymnasium as kym
from typing import Any, Dict


'''
kymnasium.Agent를 상속하여
자신만의 에이전트를 구현
'''
class YourAgent(kym.Agent):
    def act(self, observation: Any, info: Dict):
        pass

     @classmethod
    def load(cls, path: str) -> 'kym.Agent':
        pass

    def save(self, path: str):
        pass


def train():
    '''
    Killer Bill 환경은 다음과 같이 생성
    '''
    env = gym.make(
        id='kymnasium/BulletBill-Continuous-Hard-Stage-2',
        render_mode='human', # or "rgb_array"
        bgm=True # or False
     )
    '''
    여기서부터는 이 환경에 대해서 에이전트를 훈련시키는 코드를
    자유롭게 작성
    '''
```

Dependencies

에이전트 구현 시에 사용한 의존성 라이브러리의 목록을 제출해야 합니다.